GitHub Actions — 3–4 YOE Scenario-Based Interview Q&A

1. GitHub Actions workflow suddenly started failing, but it was working yesterday. What will you check?
Answer:
First, I check the failed job and workflow logs to understand the exact error. Then I compare the current run with the last successful run and check what changed recently, such as code, workflow YAML, actions versions, runner, secrets, dependencies, or external services.
I also verify whether the GitHub-hosted or self-hosted runner is healthy and whether there are any resource or connectivity issues.
Based on the error, I identify the root cause, fix it, and rerun the workflow. After the rerun, I verify that all jobs complete successfully.
Simple ga gurthupettuko:
Ninna workflow work ayindi, ivala fail ayindi ante first failed job and logs chudali.
Last successful run tho compare chesi em change ayyindo find chesi fix cheyyali.
Eng: First I check where the workflow failed and understand the error. Then I compare it with the last successful run, find what changed, fix the issue, and rerun the workflow.
Example:
GitHub → Actions → Failed workflow run → Failed job → Logs open chesi exact error ekkada vachindo chudu.
GitHub → Actions → Previous successful run → Failed run tho steps and logs compare cheyyi.
git log -5 --oneline → Recent code changes em unnayo chudu.
git diff <old_commit> <new_commit> → Recent code difference identify cheyyi.
Workflow YAML lo uses: versions → Action version change ayyinda check cheyyi.
Runner status → Runner available and healthy ga undho verify cheyyi.
Issue code/config/runner ayithe → root cause fix chesi → **Re-run jobs** → workflow successful ga complete ayyindo verify cheyyi.

---

2. GitHub Actions workflow is failing only on one runner. What will you check?
Answer:
First, I confirm that the same workflow works successfully on other runners. Then I compare the failing runner with a working runner for OS version, installed tools, environment variables, permissions, disk space, memory, and network connectivity.
I also check whether the runner has the correct labels and whether required software versions are available.
After identifying the difference, I fix the runner issue and rerun the workflow.
Simple ga gurthupettuko:
Oka runner lo matrame fail ayithe, first aa runner problem undemo chudali.
Working runner tho OS, tools, permissions, network, and resources compare cheyyali.
Eng: First I compare the failing runner with a working runner and check the tools, permissions, network, and resources.
Example:
GitHub → Actions → Failed job → Runner name check cheyyi.
uname -a → Runner OS/kernel details chudu.
df -h → Disk space check cheyyi.
free -m → Memory usage check cheyyi.
docker --version → Docker available and correct version lo undho check cheyyi.
java -version → Java version verify cheyyi if required.
which <command> → Required command installed undho chudu.
Issue runner configuration ayithe → runner fix/reconfigure chesi → workflow rerun cheyyi.

---

3. GitHub Actions workflow is stuck waiting for a runner. What will you do?
Answer:
First, I check whether a runner matching the required labels is available. Then I verify whether the runner is offline, busy, or unavailable.
For self-hosted runners, I check the runner service, machine health, network connectivity, and runner registration.
After making the required runner available, I rerun or allow the queued workflow to continue.
Simple ga gurthupettuko:
Runner kosam wait chesthe required runner available ga undho chudali.
Runner offline aa, busy aa, labels correct ga unnaya ani check cheyyali.
Eng: First I check the required runner and labels, then I verify whether the runner is offline, busy, or unavailable.
Example:
GitHub → Settings → Actions → Runners → Runner status chudu.
Workflow YAML → runs-on: → Required label correct ga undho verify cheyyi.
Self-hosted runner lo ./run.sh or configured runner service → runner running lo undho check cheyyi.
systemctl status actions.runner.* → Runner service status check cheyyi if configured as a service.
ping <github-endpoint> → Network connectivity verify cheyyi.
Runner online and label correct ayyaka → workflow **Re-run** cheyyi.

---

4. GitHub Actions has many workflow runs waiting in the queue. What will you investigate?
Answer:
I check whether enough runners are available to execute the jobs. Then I verify runner labels, concurrency settings, running workflows, and whether a specific runner is overloaded or unavailable.
I also check whether concurrency controls are intentionally limiting the number of runs.
After identifying the bottleneck, I increase runner capacity or correct the configuration.
Simple ga gurthupettuko:
Queue lo workflows ekkuva unte first runners saripothunnaya chudali.
Runner busy/offline or concurrency limit valla wait avutunnaya check cheyyali.

Eng: First I check runner availability and concurrency limits, then I identify why workflows are waiting.
Example:
GitHub → Actions → Queued workflow → Waiting reason chudu.
GitHub → Settings → Actions → Runners → Available runners count check cheyyi.
Workflow YAML → concurrency: → Queue ni limit chestunda chudu.
runs-on: → Correct runner label use chestunnara verify cheyyi.
Self-hosted runner machine → CPU/memory usage check cheyyi.
Capacity issue ayithe → additional runner add cheyyi or concurrency configuration correct cheyyi → queued workflow run avutunda verify cheyyi.

---

5. GitHub Actions workflow suddenly became very slow. What will you check?
Answer:
First, I compare the current workflow duration with previous successful runs and identify which job or step became slow.
Then I check dependency downloads, Docker builds, tests, network calls, caching, runner resources, and external services related to that step.
I improve the slow area without affecting the correctness of the pipeline and rerun it to confirm the improvement.
Simple ga gurthupettuko:
Workflow slow ayithe first ye job/step ekkuva time tiskuntundo find cheyyali.
Aa step lo dependencies, Docker, tests, network, cache, or resources problem undho check cheyyali.
Eng: First I identify the slow job or step, then I check dependencies, caching, resources, and external services.
Example:
GitHub → Actions → Workflow run → Job details → Each step duration compare cheyyi.
time npm install → Dependency installation time chudu.
time mvn test → Maven test duration check cheyyi.
docker build . → Docker build time observe cheyyi.
Workflow YAML → actions/cache → Cache configured undho verify cheyyi.
df -h → Runner disk space check cheyyi.
Slow step identify ayyaka → cache/parallelization/configuration optimize chesi → workflow rerun cheyyi.

---

6. How would you optimize a GitHub Actions workflow that takes 40 minutes?
Answer:
I first identify the jobs and steps consuming most of the execution time. Then I run independent jobs in parallel, use dependency caching, optimize Docker builds, remove unnecessary steps, and use suitable runners.
I also avoid rebuilding or downloading the same dependencies repeatedly.
After optimization, I compare the new execution time with the previous workflow.
Simple ga gurthupettuko:
40 minutes workflow ayithe first ekkada ekkuva time pothundo find cheyyali.
Independent jobs parallel ga run chesi cache use chesi unnecessary work remove cheyyali.
Eng: First I find the slow jobs, then I parallelize independent work, improve caching, and remove unnecessary processing.
Example:
GitHub Actions YAML → Independent jobs ki needs: remove chesi parallel execution enable cheyyi.
actions/cache@v4 → Maven/npm/pip dependencies cache cheyyi.
docker/build-push-action → Docker build caching enable cheyyi.
time <build-command> → Build duration compare cheyyi.
Repeated installation steps → Common unnecessary steps remove cheyyi.
Optimization tarvatha → workflow rerun chesi → total duration previous run tho compare cheyyi.

---

7. How do you implement parallel jobs in GitHub Actions?
Answer:
I identify jobs that do not depend on each other and allow them to run independently. For example, unit tests, linting, and security checks can often run in parallel.
For jobs that depend on another job, I use needs to maintain the required order.
This reduces overall workflow execution time.
Simple ga gurthupettuko:
Dependency leni jobs parallel ga run cheyyachu.
Dependency unna jobs ki needs use chesi sequential ga run cheyyali.
Eng: I run independent jobs in parallel and use needs for jobs that have dependencies.
Example:
Workflow YAML → build, test, lint independent jobs ga define cheyyi.
needs: build → Build complete ayyaka dependent job run cheyyi.
GitHub → Actions → Workflow graph → Jobs parallel ga execute avutunnaya chudu.
Job dependency correct ga undho verify chesi → workflow rerun cheyyi.

---

8. GitHub Actions secrets are failing. How will you troubleshoot?
Answer:
First, I verify that the secret exists and that the workflow is referencing the correct secret name.
Then I check whether the secret is configured at repository, environment, or organization level and whether the job has access to it.
I also verify permissions and whether the target system accepts the credential.
After correcting the secret or access configuration, I rerun the workflow.

Simple ga gurthupettuko:
Secret fail ayithe first secret name and existence check cheyyali.
Tarvatha repository/environment/organization access and target authentication check cheyyali.
Eng: First I verify the secret name and availability, then I check its scope, permissions, and target authentication.
Example:
GitHub → Settings → Secrets and variables → Actions → Secret exists undho chudu.
Workflow YAML → ${{ secrets.MY_SECRET }} → Secret name exact ga undho verify cheyyi.
Environment → Required environment secret configured undho chudu.
Workflow → Correct environment: use chestunda verify cheyyi.
Target system → Credentials valid aa ani authentication test cheyyi.
Issue secret configuration ayithe → correct secret update chesi → workflow rerun cheyyi.

---

9. A secret is visible in GitHub Actions logs. What will you do?
Answer:
I treat it as a security incident and immediately rotate or revoke the exposed secret.
Then I identify why the secret was printed, remove the exposure from the workflow or scripts, and check whether the secret appears in artifacts or other logs.
I update the workflow to use GitHub Actions secrets correctly and rerun it safely.
Simple ga gurthupettuko:
Secret log lo kanipisthe first aa secret ni rotate or revoke cheyyali.
Tarvatha secret enduku print ayyindo fix chesi other logs/artifacts kuda check cheyyali.
Eng: First I rotate the exposed secret, then I remove the exposure and check for further leaks.
Example:
GitHub → Settings → Secrets and variables → Actions → Exposed secret rotate/revoke cheyyi.
Workflow YAML → echo $SECRET / debug commands → Secret print chestunnaya remove cheyyi.
grep -R "SECRET_VALUE" . → Repository/workspace lo accidentally stored unda check cheyyi.
GitHub Actions logs → Other runs/artifacts lo exposure unda review cheyyi.
New secret configure chesi → workflow rerun chesi → logs lo secret expose avvatledu ani verify cheyyi.

---

10. GitHub Actions workflow is not triggering after a Git push. How will you troubleshoot?
Answer:
First, I check the workflow trigger configuration in the YAML file. Then I verify the branch, event type, path filters, workflow file location, and repository Actions settings.
I also check whether Actions are enabled and whether the push actually matched the configured trigger.
After correcting the trigger configuration, I push a test change and verify that the workflow starts automatically.
Simple ga gurthupettuko:
Push chesina workflow trigger avvakapothe first on: configuration check cheyyali.
Branch/path filters, Actions settings, and workflow file location verify cheyyali.
Eng: First I check the workflow trigger, branch and path filters, then I verify repository Actions settings and test another push.
Example:
Workflow YAML → on: push → Push trigger configured undho chudu.
on: push: branches: → Correct branch configured undho verify cheyyi.
paths: → Changed file path filter valla workflow skip ayyinda chudu.
GitHub → Actions → Workflow → Workflow enabled undho check cheyyi.
.github/workflows/<workflow>.yml → File correct location lo undho verify cheyyi.
git push origin <branch> → Test push chesi → GitHub Actions lo new run create ayyindo verify cheyyi.

---

11. GitHub Actions workflow works manually but fails when triggered by a push. Why?
Answer:
First, I compare the manual workflow execution with the push-triggered execution. I check whether both runs are using the same branch, inputs, environment variables, secrets, permissions, and runner.
I also verify whether the push event is triggering the expected workflow file and whether branch or path filters are changing the execution behavior.
After identifying the difference, I fix the workflow configuration and test it again with a real push.
Simple ga gurthupettuko:
Manual work ayyi push tho fail ayithe rendu runs madhya difference chudali.
Branch, secrets, inputs, permissions, runner, and workflow trigger compare cheyyali.
Eng: I compare manual and push-triggered runs and identify differences in branch, inputs, secrets, permissions, and environment.
Example:
GitHub → Actions → Manual run → Job logs save chesi → Push-triggered run logs tho compare cheyyi.
Workflow YAML → workflow_dispatch: and push: → Both triggers correct ga configured unnaya chudu.
git branch --show-current → Expected branch lo work chestunnava verify cheyyi.
Workflow → ${{ github.ref }} → Trigger ayina branch ento check cheyyi.
${{ secrets.MY_SECRET }} → Required secret available undho verify cheyyi.
Issue trigger/config ayithe → workflow fix chesi → test commit push chesi → new run successful ga execute ayyindo verify cheyyi.

12. GitHub Actions runner workspace is causing build failures. What will you check?
Answer:
First, I check whether old files from previous jobs are affecting the current build. Then I check disk space, permissions, temporary files, and whether multiple jobs are using the same workspace.
For self-hosted runners, I also verify whether the workspace is being cleaned correctly between jobs.
After cleaning or isolating the workspace, I rerun the workflow and verify the build.
Simple ga gurthupettuko:
Workspace problem ayithe old files or previous job interfere avutundha chudali.
Disk, permissions, temporary files, and cleanup check chesi required ayithe workspace clean cheyyali.
Eng: I check old files, permissions, disk space, and workspace cleanup, then clean or isolate the workspace if required.
Example:
Workflow YAML → actions/checkout@v4 → Current source correct ga checkout avutunda chudu.
pwd → Current workspace path check cheyyi.
ls -la → Old files unnaya inspect cheyyi.
df -h → Disk space check cheyyi.
du -sh * → Workspace lo ekkuva space deniki use avutundo chudu.
rm -rf <unnecessary-files> → Safe ga unnecessary files clean cheyyi.
Cleanup tarvatha → workflow rerun chesi → build successfully complete ayyindo verify cheyyi.

---

13. GitHub Actions self-hosted runner suddenly went offline during a build. What will you do?
Answer:
First, I check whether the runner machine is running and reachable. Then I check network connectivity, disk space, CPU, memory, runner service, and runner logs.
I also verify whether the runner process stopped or the machine lost connectivity with GitHub.
After restoring the runner, I rerun the failed job and verify that the runner remains online.
Simple ga gurthupettuko:
Runner offline ayithe first machine and network check cheyyali.
Tarvatha runner service, CPU, memory, disk, and logs check cheyyali.
Eng: First I check the runner machine and network, then I verify the runner service and system resources.
Example:
GitHub → Settings → Actions → Runners → Runner status chudu.
systemctl status actions.runner.* → Runner service running lo undho check cheyyi.
journalctl -u actions.runner.* -n 100 --no-pager → Runner-side errors chudu.
ping <network-endpoint> → Network connectivity check cheyyi.
df -h → Disk space verify cheyyi.
free -m → Memory usage check cheyyi.
Issue network/service/resource ayithe → fix chesi → runner online status verify cheyyi → failed job rerun cheyyi.

---

14. GitHub Actions runner CPU usage is very high. How will you investigate?
Answer:
I identify which process is consuming the CPU and determine which workflow or job is responsible. Then I check running jobs, parallel workloads, Docker processes, tests, and overall system resources.
If one job is consuming excessive resources, I optimize it or move the workload to a more suitable runner.
After the change, I monitor CPU usage and verify workflow performance.
Simple ga gurthupettuko:
CPU high ayithe first ye process and ye job CPU use chestundo find cheyyali.
Workload ekkuva unte optimize cheyyali or suitable runner ki move cheyyali.
Eng: First I identify the process and workflow consuming CPU, then I optimize or distribute the workload.
Example:
top → CPU ekkuva use chestunna process identify cheyyi.
ps aux --sort=-%cpu | head → Top CPU processes chudu.
docker stats → Docker containers CPU usage check cheyyi.
GitHub → Actions → Running jobs → Heavy job identify cheyyi.
Workflow YAML → Parallel jobs/workload check cheyyi.
CPU-heavy workload ayithe → optimize or larger runner use chesi → workflow rerun chesi CPU usage verify cheyyi.

---

15. GitHub Actions runner disk space is full. How will you troubleshoot?
Answer:
First, I identify what is consuming the disk, such as workspace files, Docker images, build artifacts, logs, caches, or temporary files.
Then I safely remove unnecessary data and configure appropriate cleanup or retention.
For self-hosted runners, I also make sure disk cleanup is automated so the problem does not repeatedly occur.
Simple ga gurthupettuko:
Disk full ayithe first ekkuva space deniki use avutundo find cheyyali.
Unnecessary files/cache/Docker data clean chesi future cleanup automate cheyyali.
Eng: First I find what is consuming disk space, clean unnecessary data, and configure proper cleanup.
Example:
df -h → Disk full ayinda check cheyyi.
du -sh /actions-runner/* → Runner directory space usage chudu.
docker system df → Docker images/containers space check cheyyi.
docker image prune → Unused Docker images safe ga clean cheyyi when appropriate.

GitHub → Actions → Caches → Unnecessary cache identify cheyyi.
Cleanup tarvatha → df -h → Free space verify cheyyi → workflow rerun cheyyi.

---

16. Build is successful but deployment fails in GitHub Actions. What will you check?
Answer:
Since the build is successful, I focus on the deployment stage. I check the artifact or Docker image, deployment credentials, target connectivity, environment configuration, permissions, deployment commands, and target-side logs.
I also verify whether the same artifact produced by the build is being deployed.
After identifying the deployment issue, I fix it and rerun only the required deployment stage when possible.
Simple ga gurthupettuko:
Build success ayithe build ni malli check cheyyakunda deployment side chudali.
Artifact, credentials, connectivity, config, permissions, and target logs verify cheyyali.
Eng: Since the build passed, I focus on the deployment environment, artifact, credentials, configuration, and target logs.
Example:
GitHub → Actions → Successful build job → Artifact name/version note chesuko.
docker images → Expected image available undho check cheyyi.
docker pull <image>:<tag> → Registry image accessible undho verify cheyyi.
kubectl get pods -n <namespace> → Target deployment status chudu.
kubectl describe deployment <deployment> -n <namespace> → Deployment error/events chudu.
kubectl logs <pod> -n <namespace> → Application-side deployment issue check cheyyi.
Issue fix chesi → deployment job rerun cheyyi → application health verify cheyyi.

---

17. GitHub Actions deployed an old Docker image instead of the latest one. What will you check?
Answer:
I trace the image version from the Git commit to the GitHub Actions run, Docker build, image tag, registry, deployment configuration, and running Pod or container.
I also check whether a mutable tag such as latest is causing confusion or whether an old image is being cached.
After identifying the incorrect version, I update the tagging or deployment configuration and redeploy.
Simple ga gurthupettuko:
Old image vaste Git nunchi running container varaku version trace cheyyali.
Tag, registry, deployment config, and running image correct ga unnaya chudali.
Eng: I trace the image version from Git to the running container and verify the tag and deployment configuration.
Example:
git rev-parse --short HEAD → Current commit ID chudu.
GitHub Actions → Build logs → Image tag ento verify cheyyi.
docker images | grep <image> → Local image tag check cheyyi.
docker pull <registry>/<image>:<tag> → Registry lo expected image undho verify cheyyi.
kubectl get deployment <deployment> -o jsonpath='{.spec.template.spec.containers[*].image}' → Deployment image check cheyyi.
kubectl get pods -o wide → Running Pods identify cheyyi.
Wrong tag ayithe → unique image tag use chesi → deployment rerun cheyyi → running image verify cheyyi.

---

18. How do you rollback a failed Production deployment using GitHub Actions?
Answer:
First, I identify the last known-good version from the deployment history or release information.
Then I deploy that known-good artifact or trigger the approved rollback workflow.
After rollback, I verify application health, Pods, Services, and monitoring.
Once Production is stable, I investigate why the new release failed.
Simple ga gurthupettuko:
Production deployment fail ayithe first last working version identify cheyyali.
Aa version ki rollback chesi application healthy ga undho verify cheyyali.
Eng: I identify the last good version, roll back to it, and verify the application before investigating the failed release.
Example:
GitHub → Releases/Actions → Last successful version identify cheyyi.
git log --oneline -10 → Recent release commits chudu.
kubectl rollout history deployment/<deployment> -n <namespace> → Deployment history check cheyyi.
kubectl rollout undo deployment/<deployment> -n <namespace> → Previous Kubernetes revision ki rollback cheyyi.
kubectl rollout status deployment/<deployment> -n <namespace> → Rollback successful aa check cheyyi.
kubectl get pods -n <namespace> → Pods healthy ga unnaya verify cheyyi.

---

19. How do you prevent accidental Production deployment in GitHub Actions?
Answer:
I protect Production using GitHub Environments, required approvals, restricted secrets, protected branches, and environment-specific permissions.
I also make sure Production deployment jobs run only from approved branches or release tags.
This ensures that a developer cannot accidentally deploy an unapproved change directly to Production.
Simple ga gurthupettuko:
Production ki direct deployment access andariki ivvakudadhu.
Environment approval, protected branch, restricted secrets, and approved version use cheyyali.
Eng: I protect Production with approvals, restricted secrets, protected branches, and controlled deployment conditions.
Example:
GitHub → Settings → Environments → production → Required reviewers configure cheyyi.

Workflow YAML → environment: production → Production environment protection apply cheyyi.
Workflow YAML → if: github.ref == 'refs/heads/main' → Only approved branch deployment allow cheyyi.
GitHub → Settings → Branches → Branch protection enable cheyyi.
Production secrets → Production environment lo separate ga maintain cheyyi.
Approval tarvatha → deployment job execute ayyi → Production health verify cheyyi.

---

20. How do you design one GitHub Actions workflow for DEV, QA, UAT, and PROD?
Answer:
I create a reusable workflow and keep environment-specific values such as credentials, URLs, namespaces, and configuration separate.
I use GitHub Environments to manage environment-specific secrets and approvals.
I promote the same tested artifact through DEV, QA, UAT, and PROD instead of rebuilding it for every environment.
This gives consistency and reduces deployment differences.
Simple ga gurthupettuko:
One workflow ni different environments ki reuse cheyyachu.
Environment config separate ga petti same tested artifact ni promote cheyyali.
Eng: I use one reusable workflow, separate environment configuration, and promote the same tested artifact across environments.
Example:
GitHub → Settings → Environments → dev, qa, uat, production create cheyyi.
Workflow YAML → environment: dev → DEV deployment configure cheyyi.
environment: qa → QA deployment configure cheyyi.
environment: production → PROD approval and secrets use cheyyi.
docker build -t app:${{ github.sha }} . → Commit-based immutable image create cheyyi.
DEV lo test ayina same image tag → QA/UAT/PROD ki promote cheyyi.
Each environment deployment tarvatha → health check run chesi → next environment ki promote cheyyi.

---

21. How do you manage different environment credentials in GitHub Actions?
Answer:
I maintain separate credentials for DEV, QA, UAT, and Production using GitHub Environments or appropriate repository and organization secrets.
Each deployment job receives only the credentials required for that environment.
Production credentials should not be exposed to lower environments, and secrets should never be hardcoded in the workflow.
Simple ga gurthupettuko:
DEV, QA, UAT, PROD credentials separate ga maintain cheyyali.
Pipeline ki required environment credential matrame access ivvali.
Eng: I keep environment credentials separate and provide each job only the access required for that environment.
Example:
GitHub → Settings → Environments → dev → DEV secrets configure cheyyi.
qa → QA credentials configure cheyyi.
production → PROD credentials configure cheyyi.
Workflow YAML → environment: production → Correct environment secrets access cheyyi.
${{ secrets.AWS_ACCESS_KEY_ID }} → Secret ni workflow lo securely reference cheyyi.
echo $AWS_SECRET_ACCESS_KEY → Ila secret ni print cheyyakudadhu.
Deployment tarvatha → logs check chesi secrets expose avvaledu ani verify cheyyi.

---

22. How do you use reusable workflows in a real GitHub Actions project?
Answer:
I move common CI/CD logic into reusable workflows so multiple repositories or applications can follow the same process.
For example, build, test, Docker image creation, security scanning, and deployment logic can be centralized.
Individual workflows then provide only project-specific inputs such as application name, environment, or image tag.
This reduces duplicate YAML and makes pipeline maintenance easier.
Simple ga gurthupettuko:
Same CI/CD logic multiple repositories ki unte reusable workflow create cheyyali.
Common logic oka place lo maintain chesi different projects nunchi call cheyyali.
Eng: I centralize common CI/CD logic in reusable workflows and call them from project-specific workflows.
Example:
.github/workflows/reusable-build.yml → Common build workflow create cheyyi.
on: workflow_call: → Reusable workflow ga configure cheyyi.
Caller workflow lo uses: ./.github/workflows/reusable-build.yml → Reusable workflow call cheyyi.
with: → Application-specific inputs pass cheyyi.
secrets: → Required secrets securely pass cheyyi.
Common logic change chesi → dependent workflows run chesi → all repositories lo expected behavior verify cheyyi.

---

23. Why are reusable workflows useful in GitHub Actions?
Answer:
If the same workflow logic is copied into many repositories, maintaining it becomes difficult.
Reusable workflows allow common logic to be maintained in one place and reused across projects.
This improves consistency, reduces duplicate code, and makes updates easier to control.
Simple ga gurthupettuko:
Same YAML chala repositories lo copy chesthe maintenance kastam.
Reusable workflow tho common logic oka place lo maintain chesi multiple projects ki reuse cheyyachu.

Eng: Reusable workflows reduce duplicate YAML and allow common CI/CD logic to be maintained centrally.
Example:
grep -R "docker build" .github/workflows/ → Duplicate Docker build logic identify cheyyi.
Common steps → Reusable workflow lo move cheyyi.
on: workflow_call: → Workflow ni reusable ga define cheyyi.
Project workflows → Reusable workflow ni uses: tho call cheyyi.
Common workflow lo change chesi → multiple repositories runs verify cheyyi.

---

24. A GitHub Action version update broke an existing workflow. What will you do?
Answer:
First, I identify which Action version was changed and check the workflow error and release notes.
Then I compare the previous working version with the new version and determine whether there are breaking changes or input/output changes.
If the new version is causing the issue, I can temporarily pin the workflow to the previous compatible version while testing the fix.
After validation, I update the Action safely.
Simple ga gurthupettuko:
Action update tarvatha workflow fail ayithe changed Action ni identify cheyyali.
Logs and version changes check chesi required ayithe previous working version ki pin cheyyali.
Eng: I identify the changed Action, confirm the issue, temporarily use a compatible version if needed, and then apply the proper fix.
Example:
Workflow YAML → uses: actions/checkout@v4 → Action version identify cheyyi.
Git diff → Workflow lo recently changed Action version chudu.
GitHub → Actions → Failed run → Exact error inspect cheyyi.
Previous working commit → Old uses: version identify cheyyi.
git checkout <previous_commit> -- .github/workflows/<workflow>.yml → Previous workflow configuration compare cheyyi.
Compatible version pin chesi → workflow rerun cheyyi → success confirm ayyaka controlled upgrade cheyyi.

---

25. How do you safely update GitHub Actions used in Production?
Answer:
I first review the Action version changes, compatibility, inputs, outputs, and security implications.
Then I test the updated workflow in a lower environment or test branch.
I keep the previous working version available so that I can quickly revert if the update causes a problem.
Only after successful validation do I apply the change to Production workflows.
Simple ga gurthupettuko:
Production workflow lo Action ni direct ga update cheyyakudadhu.
First test cheyyali, previous version ready ga unchali, tarvatha controlled ga update cheyyali.
Eng: I review and test the Action update first, keep the previous version available, and then perform a controlled Production update.
Example:
Workflow YAML → Current Action version note chesuko.
git checkout -b action-upgrade-test → Separate test branch create cheyyi.
Action version update chesi → workflow run cheyyi.
GitHub → Actions → Test results/logs verify cheyyi.
Previous version Git lo maintain cheyyi.
Test successful ayyaka → PR create chesi → review/approval tarvatha Production branch ki merge cheyyi.

---

26. GitHub Actions self-hosted runner is using very high CPU or memory. What will you check?
Answer:
I check which process or workflow is consuming the resources and whether multiple heavy jobs are running on the same runner.
Then I review Docker builds, tests, parallel jobs, runner configuration, and available system resources.
If required, I reduce concurrency or move heavy workloads to larger or additional runners.
After the change, I monitor resource usage and workflow performance.
Simple ga gurthupettuko:
Runner CPU/memory high ayithe heavy jobs ekkuva run avutunnaya chudali.
Concurrency reduce cheyyali or heavy workloads ni separate/larger runners ki move cheyyali.
Eng: I identify resource-heavy jobs and reduce contention by optimizing or distributing workloads across runners.
Example:
top → CPU/memory consuming process identify cheyyi.
free -m → Memory usage check cheyyi.
docker stats → Docker workload resources chudu.
GitHub → Actions → Running jobs → Heavy workflow identify cheyyi.
Workflow concurrency: → Too many jobs run avutunnaya check cheyyi.
Heavy workload ayithe → additional/larger runner use chesi → workflow rerun cheyyi → CPU/memory usage verify cheyyi.

---

27. How do you control the number of concurrent GitHub Actions jobs?
Answer:
I consider runner capacity, CPU, memory, workload type, and deployment requirements before allowing high concurrency.
For jobs that must not run simultaneously, I use GitHub Actions concurrency controls.
For example, Production deployments can be restricted so that only one deployment runs at a time.
This prevents resource contention and deployment conflicts.

Simple ga gurthupettuko:
Concurrency ni runner resources and workload batti control cheyyali.
Production deployments conflict kakunda concurrency use cheyyali.
Eng: I control concurrency based on runner capacity and use concurrency groups to prevent conflicting deployments.
Example:
Workflow YAML → concurrency: → Concurrency group configure cheyyi.
group: production-deployment → PROD deployments same group lo control cheyyi.
cancel-in-progress: false → Existing Production deployment ni accidentally cancel kakunda configure cheyyi.
GitHub → Actions → Multiple runs → Queue/cancel behavior verify cheyyi.
Production lo → One deployment at a time execute avutunda verify cheyyi.

---

28. Two GitHub Actions jobs are interfering with each other. What could be the reason?
Answer:
I check whether both jobs are using the same workspace, files, ports, temporary directories, deployment environment, or external resources.
On self-hosted runners, shared filesystem resources can especially cause conflicts.
I isolate the jobs, use separate resources, or control concurrency where required.
Then I rerun the workflow and verify that both jobs execute independently.
Simple ga gurthupettuko:
Rendu jobs interfere ayithe shared resource emaina undha chudali.
Workspace, files, ports, temp directories, or deployment target conflict unte separate cheyyali.
Eng: I check for shared resources and isolate jobs or control concurrency to prevent conflicts.
Example:
pwd → Both jobs workspace path compare cheyyi.
ls -la → Shared files unnaya check cheyyi.
ss -lntp → Same port use chestunnaya chudu.
Workflow YAML → Same deployment environment target chestunnaya verify cheyyi.
concurrency: → Conflicting deployments ni control cheyyi.
Separate workspace/resource configure chesi → workflow rerun chesi → jobs independently successful ga complete ayyaya verify cheyyi.

---

29. GitHub Actions dependency download is failing. What will you check?
Answer:
First, I check whether the package or artifact repository is reachable and whether DNS and network connectivity are working.
Then I verify proxy settings, repository credentials, package-manager configuration, and whether the required dependency version exists.
I also check whether a recent repository or dependency change caused the issue.
After fixing the connectivity or dependency problem, I rerun the workflow.
Simple ga gurthupettuko:
Dependency download fail ayithe first repository reachable ga undho chudali.
Network, DNS, proxy, credentials, package configuration, and dependency version check cheyyali.
Eng: I check repository access, network, proxy, credentials, package configuration, and dependency availability.
Example:
curl -I <repository-url> → Package repository reachable ga undho check cheyyi.
nslookup <repository-host> → DNS resolution verify cheyyi.
ping <repository-host> → Network connectivity check cheyyi when ICMP is allowed.
npm config list → npm configuration verify cheyyi.
mvn help:effective-settings → Maven repository configuration check cheyyi.
pip config list → Python package configuration verify cheyyi.
Required dependency version repository lo undho chusi → config fix chesi → workflow rerun cheyyi.

---

30. GitHub Actions workflow works on one branch but fails on another. What will you compare?
Answer:
I compare the two branches for source-code changes, workflow YAML changes, dependency versions, environment configuration, secrets, parameters, and branch-specific conditions.
I also verify whether the workflow file itself is different between the branches.
This helps determine whether the issue is caused by application code or CI/CD configuration.
After identifying the difference, I fix the correct branch and rerun the workflow.
Simple ga gurthupettuko:
Oka branch work, another fail ayithe rendu branches madhya differences chudali.
Code, workflow YAML, dependencies, parameters, secrets, and branch conditions compare cheyyali.
Eng: I compare the branches for code, workflow configuration, dependencies, parameters, and environment differences.
Example:
git diff main..feature-branch -- .github/workflows/ → Workflow YAML differences chudu.
git diff main..feature-branch → Code/configuration changes identify cheyyi.
git log main..feature-branch --oneline → Branch-specific commits chudu.
GitHub → Actions → Both branch runs → Failed step compare cheyyi.
${{ github.ref }} → Workflow ye branch nunchi run avutundo verify cheyyi.
Difference identify ayyaka → required branch lo fix chesi → workflow rerun cheyyi → successful result verify cheyyi.

31. GitHub Actions workflow is taking too long. What will you check?
Answer:
First, I identify which job or step is consuming most of the execution time. Then I check dependency installation, Docker builds, test execution, artifact uploads, and unnecessary repeated steps.
I also verify whether caching is configured correctly and whether independent jobs can run in parallel.
Based on the bottleneck, I optimize the workflow and rerun it to compare the execution time.

Simple ga gurthupettuko:
Workflow slow ayithe → first ye job/step ekkuva time tiskuntundo chudu → dependencies, Docker, tests, artifacts check cheyyi → caching and parallel jobs use cheyyi → optimize chesi runtime verify cheyyi.
Eng: First I identify the slow job or step, check dependencies and resource usage, improve caching or parallel execution, and verify the reduced runtime.
Example:
GitHub → Actions → Failed/Slow workflow → Job open chesi each step execution time chudu.
git log -5 --oneline → Recent code changes em unnayo check cheyyi.
docker images → Unnecessary Docker images or build-related issues unnaya chudu.
du -sh ./* → Workspace lo ekkuva space ekkada use avutundo check cheyyi.
time npm ci / time mvn clean install → Dependency/build step ekkuva time tiskuntunda check cheyyi.
GitHub Actions cache → Dependencies repeatedly download avutunnaya verify cheyyi.
Independent jobs unte → needs: dependency remove chesi parallel ga run cheyyagalama check cheyyi.
Issue identify ayyaka → workflow optimize chesi → Actions lo workflow rerun chesi → previous runtime tho compare cheyyi.

---

32. GitHub Actions workflow is failing only on one runner. What will you check?
Answer:
First, I confirm that the same workflow works successfully on other runners. Then I compare the failing runner with a working runner for OS version, installed tools, environment variables, permissions, disk space, CPU, memory, and network connectivity.
I also verify whether the runner has the required labels and dependencies.
After identifying the runner-specific issue, I fix it and rerun the workflow.
Simple ga gurthupettuko:
Oka runner lo matrame fail ayithe → working runner tho compare cheyyi → OS, tools, permissions, network, resources check cheyyi → difference fix chesi rerun cheyyi.
Eng: If the workflow fails only on one runner, I compare it with a working runner, identify the environment difference, fix it, and rerun the workflow.
Example:
GitHub → Settings → Actions → Runners → Failing runner status Online/Offline chudu.
uname -a → Runner OS and kernel details check cheyyi.
df -h → Disk space saripothunda check cheyyi.
free -m → Memory availability check cheyyi.
java -version / node --version / docker --version → Required tools installed and correct versions lo unnaya chudu.
echo $PATH → Required binaries PATH lo unnaya check cheyyi.
systemctl status actions.runner.* → Self-hosted runner service running lo undho check cheyyi.
Issue runner configuration ayithe → fix chesi → GitHub Actions workflow rerun cheyyi → successful ga execute avutunda verify cheyyi.

---

33. GitHub Actions job is stuck waiting for a runner. What will you check?
Answer:
First, I check whether a runner matching the required labels is available. Then I verify whether the runner is offline, busy, or has reached its execution capacity.
I also check the workflow runs-on configuration and repository or organization runner availability.
After correcting the runner configuration or availability issue, I rerun the workflow.
Simple ga gurthupettuko:
Runner kosam wait chesthe → runs-on label check cheyyi → matching runner available aa chudu → runner offline/busy aa check cheyyi → issue fix cheyyi.
Eng: I verify the required runner label, availability, runner status, and execution capacity, then fix the configuration if required.
Example:
GitHub → Actions → Workflow run → Job → Waiting for a runner message chudu.
GitHub → Settings → Actions → Runners → Available runners check cheyyi.
Workflow YAML → runs-on: → Correct runner label use chestunnama verify cheyyi.
./run.sh → Self-hosted runner required ayithe runner manually start chesi check cheyyi.
systemctl status actions.runner.* → Runner service active ga undho chudu.
Runner busy/offline ayithe → available runner ki job schedule cheyyi or runner issue fix cheyyi.
Fix ayyaka → workflow rerun chesi → job runner ni successfully pick chesindo verify cheyyi.

---

34. GitHub Actions secrets are not available inside the workflow. What will you check?
Answer:
First, I verify that the secret exists at the correct repository, environment, or organization level. Then I check whether the workflow is referencing the correct secret name.
I also verify environment protection rules and whether the job is actually associated with the required environment.
I never print the secret value in logs; instead, I verify its presence through safe checks.
Simple ga gurthupettuko:
Secret dorakakapothe → secret correct place lo undha chudu → name check cheyyi → environment mapping check cheyyi → secret value ni logs lo print cheyyakudadhu.

Eng: I verify the secret location, name, environment configuration, and permissions without exposing the secret value.
Example:
GitHub → Repository → Settings → Secrets and variables → Actions → Required secret exists aa check cheyyi.
Workflow YAML → ${{ secrets.MY_SECRET }} → Secret name exact ga match avutunda chudu.
Workflow → environment: → Correct Production/QA environment assign ayyinda verify cheyyi.
GitHub → Settings → Environments → Environment protection rules check cheyyi.
echo "${{ secrets.MY_SECRET }}" → **Use cheyyakudadhu**, because secret logs lo expose avvachu.
Instead → [-n "${MY_SECRET}"] && echo "Secret is available" → Secret value expose cheyyakunda availability verify cheyyi.
Configuration fix ayyaka → workflow rerun chesi → authentication successful ga avutunda verify cheyyi.

---

35. A GitHub Actions secret was exposed in workflow logs. What will you do?
Answer:
I treat the exposed secret as compromised and immediately rotate or revoke it. Then I identify how the secret reached the logs and remove the unsafe command or configuration.
I also review workflow logs, artifacts, and repository history for additional exposure.
After rotating the secret, I update the GitHub secret and rerun the workflow using the new credential.
Simple ga gurthupettuko:
Secret logs lo expose ayithe → first rotate/revoke cheyyi → exposure reason find cheyyi → workflow fix cheyyi → new secret update cheyyi → rerun cheyyi.
Eng: I immediately rotate the exposed secret, remove the source of exposure, check for further leaks, and update the workflow with the new secret.
Example:
GitHub → Actions → Workflow run → Job logs → Secret ekkada expose ayyindo identify cheyyi.
Secret provider → Exposed credential → Immediately revoke/rotate cheyyi.
GitHub → Settings → Secrets and variables → Actions → Old secret replace chesi new secret add cheyyi.
git grep "SECRET_NAME" → Repository lo unsafe references unnaya check cheyyi.
Workflow YAML → set -x or commands printing environment → Secret exposure cause remove cheyyi.
Artifacts/log outputs → Secret accidentally store ayyinda check cheyyi.
New secret configure chesi → workflow rerun cheyyi → authentication successful ga undho verify cheyyi.

---

36. GitHub Actions workflow is not triggering after a Git push. What will you check?
Answer:
First, I check whether the workflow file exists in the expected branch and whether the on trigger configuration matches the pushed branch or event.
Then I verify repository Actions settings, workflow syntax, branch filters, and whether the push actually reached GitHub.
I also check whether the workflow is disabled or blocked by repository settings.
After correcting the trigger configuration, I push a test change and verify that the workflow starts.
Simple ga gurthupettuko:
Push tarvatha workflow trigger avvakapothe → workflow file and on: trigger check cheyyi → branch filter check cheyyi → Actions settings verify cheyyi → test push cheyyi.
Eng: I verify the workflow trigger, branch filters, repository Actions settings, and whether the push event reached GitHub.
Example:
git push origin main → Push successful ga GitHub ki vellinda check cheyyi.
.github/workflows/*.yml → Workflow file correct location lo undho chudu.
on: push: → Push trigger configured aa verify cheyyi.
branches: [main] → Correct branch filter undho check cheyyi.
GitHub → Actions → Workflow → Workflow enabled aa check cheyyi.
GitHub → Actions → Workflow runs → Recent run create ayyinda chudu.
YAML syntax issue unte → workflow syntax fix chesi → test commit push cheyyi → workflow automatically trigger ayyindo verify cheyyi.

---

37. GitHub Actions workflow works manually but does not trigger from pull requests. Why?
Answer:
First, I compare the manual trigger configuration with the pull_request trigger configuration.
Then I verify the target branches, pull request event types, workflow file location, and repository settings.
I also check whether the PR comes from a fork because security restrictions can affect secrets and workflow execution.
After fixing the trigger or permission configuration, I create or update a test pull request and verify the workflow.
Simple ga gurthupettuko:
Manual work, PR trigger fail ayithe → pull_request event check cheyyi → branch/event type verify cheyyi → fork/security settings chudu → test PR create cheyyi.
Eng: I compare manual and PR triggers, verify branch and event configuration, check permissions and fork restrictions, and test with a pull request.
Example:
.github/workflows/ci.yml → on: pull_request: configured aa check cheyyi.
branches: [main] → PR target branch correct ga undho verify cheyyi.
types: [opened, synchronize, reopened] → Required PR events configured aa chudu.
GitHub → Pull Request → Checks → Workflow trigger ayyinda check cheyyi.
PR fork nunchi vasthe → Secrets and permission restrictions check cheyyi.
Workflow permissions → permissions: configuration verify cheyyi.
Fix ayyaka → PR lo new commit push chesi → workflow automatically run ayyindo verify cheyyi.

38. GitHub Actions artifact upload is failing. What will you check?
Answer:
First, I check the artifact upload step and the exact error message. Then I verify whether the expected file or directory was actually created and whether the configured path is correct.
I also check artifact naming, file permissions, disk space, and whether the workflow is trying to upload an invalid or missing path.
After correcting the path or build step, I rerun the workflow and verify the artifact.
Simple ga gurthupettuko:
Artifact upload fail ayithe → first error chudu → file create ayyinda check cheyyi → path/name/permissions verify cheyyi → fix chesi rerun cheyyi.
Eng: I check the upload error, verify the artifact path and file existence, correct the configuration, and rerun the workflow.
Example:
GitHub → Actions → Failed workflow → Upload artifact step → Exact error chudu.
ls -lah → Artifact file actually create ayyinda check cheyyi.
find . -name "*.jar" → Expected artifact ekkada generate ayyindo chudu.
ls -ld <artifact_directory> → Directory permissions check cheyyi.
Workflow YAML → path: → Correct artifact path configured aa verify cheyyi.
df -h → Runner disk space available ga undho check cheyyi.
Path fix chesi → workflow rerun cheyyi → GitHub Actions run lo artifact successfully upload ayyindo verify cheyyi.

---

39. GitHub Actions Docker build is failing. What will you check?
Answer:
First, I check the Docker build logs to identify whether the issue is related to the Dockerfile, build context, dependencies, registry access, or runner environment.
Then I verify the Dockerfile syntax, required files, build arguments, credentials, and available disk space.
I also check whether the base image or dependencies can be downloaded successfully.
After fixing the root cause, I rerun the Docker build and verify that the image is created successfully.
Simple ga gurthupettuko:
Docker build fail ayithe → logs chudu → Dockerfile/context check cheyyi → dependencies/base image check cheyyi → credentials/resources verify cheyyi → image build cheyyi.
Eng: I inspect the Docker build error, verify the Dockerfile and build context, check dependencies and resources, fix the issue, and rebuild the image.
Example:
GitHub → Actions → Build job → Docker build step → Exact Docker error chudu.
docker version → Docker available and working ga undho check cheyyi.
docker build -t test-image . → Dockerfile locally reproduce cheyyagalama chudu.
cat Dockerfile → Dockerfile instructions verify cheyyi.
ls -lah → Required files build context lo unnaya check cheyyi.
df -h → Runner disk space check cheyyi.
docker pull <base-image> → Base image pull avutunda verify cheyyi.
Issue fix chesi → GitHub Actions workflow rerun cheyyi → Docker image successfully build ayyindo verify cheyyi.

---

40. GitHub Actions successfully builds and pushes an image, but Kubernetes deployment fails. What will you check?
Answer:
Since the image build and push are successful, I focus on the deployment side. I verify the image tag, Kubernetes credentials, cluster connectivity, namespace, manifests, Secrets, ConfigMaps, and resource configuration.
Then I check Kubernetes events and Pod logs to identify why the workload is not starting or becoming ready.
After fixing the deployment issue, I rerun the deployment and verify the rollout and application health.
Simple ga gurthupettuko:
Image push success but K8s fail ayithe → image/tag verify cheyyi → cluster access check cheyyi → YAML/Secrets/ConfigMap check cheyyi → Pods/events/logs chudu → fix and rollout verify cheyyi.
Eng: If image creation succeeds but Kubernetes deployment fails, I verify the image, cluster access, manifests, configuration, events, and Pod logs before fixing the deployment.
Example:
docker images → Expected image and tag local runner lo create ayyinda check cheyyi.
docker push <registry>/<image>:<tag> → Registry push successful aa verify cheyyi.
kubectl config current-context → Correct Kubernetes cluster context undho chudu.
kubectl get nodes → Cluster connectivity and node health check cheyyi.
kubectl get pods -n <namespace> → Deployment Pods status chudu.
kubectl describe pod <pod> -n <namespace> → Pod scheduling/image/config error identify cheyyi.
kubectl logs <pod> -n <namespace> → Application startup error chudu.
kubectl get events -n <namespace> --sort-by=.lastTimestamp → Latest Kubernetes events check cheyyi.
Issue fix chesi → kubectl rollout status deployment/<deployment> -n <namespace> → Deployment successful ga rollout ayyindo verify cheyyi.

41. How would you design a reliable GitHub Actions CI/CD pipeline for Production?
Answer:
I design the workflow with clear stages for checkout, build, unit testing, code quality, security scanning, Docker image creation, artifact or image publishing, deployment, verification, and notification.
I use reusable workflows or actions where appropriate, environment-specific secrets, protected Production environments, approvals, and versioned artifacts.
I also make the pipeline fail fast, support rollback, and provide sufficient logs and monitoring for troubleshooting.
Before Production deployment, I validate the same artifact in lower environments and promote it instead of rebuilding it.
Simple ga gurthupettuko:
GitHub Actions Production pipeline → Code → Build → Test → Quality → Security → Docker image → Registry → DEV/QA → Approval → PROD → Health check → Rollback.
Eng: I build a secure and reusable CI/CD pipeline with testing, quality and security checks, versioned artifacts, controlled Production deployment, verification, monitoring, and rollback.
Example:
git push origin main → GitHub Actions CI workflow trigger cheyyi.
npm test / mvn test → Unit tests run chesi application correct ga undho verify cheyyi.
docker build -t <registry>/<app>:<commit-sha> . → Unique versioned Docker image build cheyyi.
docker push <registry>/<app>:<commit-sha> → Image registry ki push cheyyi.
kubectl config current-context → Correct target cluster verify cheyyi.
helm upgrade --install <release> <chart> -f values-dev.yaml → DEV environment lo deploy cheyyi.
kubectl rollout status deployment/<deployment> -n <namespace> → DEV rollout successful aa verify cheyyi.
GitHub → Environments → Production → Required approval configure chesi → approved deployment matrame allow cheyyi.
helm upgrade --install <release> <chart> -f values-prod.yaml --set image.tag=<commit-sha> → Same tested image ni PROD ki deploy cheyyi.
kubectl get pods -n <namespace> → Pods healthy ga unnaya chudu.
kubectl logs <pod> -n <namespace> → Application startup/runtime errors verify cheyyi.
kubectl rollout status deployment/<deployment> -n <namespace> → Production rollout successful ga complete ayyindo verify cheyyi.
Issue vaste → previous known-good image/version or Helm revision identify chesi → controlled rollback perform chesi → application health verify cheyyi.